

HisabDo AI Copilot

Complete End-to-End AI/ML Architecture, API Design, Roadmap and Implementation Plan

Project: HisabDo Website Enhancement

Department: AI/ML

Prepared By: Taha — AI/ML Team Lead

Architecture Type: Offline-First, Multilingual, Cost-Optimized AI System

1. Executive Summary

HisabDo AI Copilot is an intelligent AI layer designed specifically for the HisabDo financial management ecosystem.

Unlike a traditional chatbot that only answers general questions, HisabDo AI Copilot is designed as a multi-module intelligent assistant capable of:

- Answering questions about HisabDo
- Understanding multiple languages
- Searching HisabDo documentation and FAQs
- Analyzing financial information safely
- Providing business insights
- Recommending relevant features
- Understanding user intent
- Supporting voice interaction
- Providing contextual responses
- Working with offline-first principles
- Reducing hallucinated financial information
- Maintaining user privacy

The architecture separates deterministic financial calculations from generative AI.

This means:

Code performs calculations.

AI understands questions and explains results.

For example:

User asks:

How much does Ali owe me?

The AI will not invent an answer.

The system will:

1. Detect the user's intent.
2. Identify the customer "Ali".
3. Retrieve the real balance from HisabDo data.
4. Calculate the balance using application logic.
5. Pass the verified result to the AI response layer.
6. Generate a natural-language answer.

This architecture improves accuracy, privacy, reliability and trust.

2. Current HisabDo Context

HisabDo is an offline-first financial management platform designed for managing:

- Khata
- Ledger
- Udhar
- Expenses
- Receivables
- Customers
- Transactions
- Analytics
- PDF Reports
- Voice Entry
- Backup and Restore

The platform also supports multiple languages and currencies.

The AI architecture must therefore enhance the existing product rather than replace its current functionality.

The proposed AI system is called:

HisabDo AI Copilot

3. Core Vision

The vision is to transform HisabDo from:

A digital ledger application

into:

An intelligent financial assistant that understands users, business records and product knowledge.

The AI system will provide three major intelligence layers:

1. **Knowledge Intelligence**
 2. **Financial Intelligence**
 3. **User Experience Intelligence**
-

4. Major AI Features

4.1 AI Chatbot

The chatbot will answer questions such as:

- What is HisabDo?
- How do I add a customer?
- How can I record a transaction?
- How can I export a PDF?
- How does offline mode work?
- How can I restore my backup?

The chatbot will use RAG instead of relying only on general LLM knowledge.

4.2 FAQ Assistant

The FAQ Assistant will search verified HisabDo content before generating an answer.

Knowledge sources may include:

- FAQs
- Documentation
- Website content
- Blog articles

- Product guides
 - Feature descriptions
 - Help articles
-

4.3 Financial AI Assistant

The Financial AI Assistant will help users understand their own financial records.

Example questions:

- How much does Ali owe me?
- Who has the highest outstanding balance?
- What were my expenses this month?
- Which customer has not paid recently?
- Compare this month's expenses with last month.
- What is my total receivable amount?

Financial values will always be retrieved or calculated using verified system data.

The LLM will not perform critical financial calculations independently.

4.4 Smart Business Insights

The system can detect useful patterns such as:

- Increasing expenses
- Outstanding customer balances
- Unusual spending patterns
- Inactive customers
- High-value customers
- Frequently used categories
- Monthly transaction trends

Example:

Your recorded expenses increased by 18% compared with the previous month.

4.5 AI Recommendations

The recommendation engine can suggest:

- Features the user may find useful
- Reports to generate

- Customers requiring follow-up
- Expense categories
- Outstanding payment reminders
- Business insights

Example:

You have three customers with long-pending balances. Consider reviewing their transaction history.

4.6 Multilingual AI

The AI Copilot will support:

- English
- Urdu
- Hindi
- Arabic
- Roman Urdu
- Extensible support for future languages

Examples:

English:

How much does Ali owe me?

Urdu:

علی کے پاس میرا کتنا بقایا ہے؟

Roman Urdu:

Ali ka mera kitna paisa reh raha hai?

Hindi:

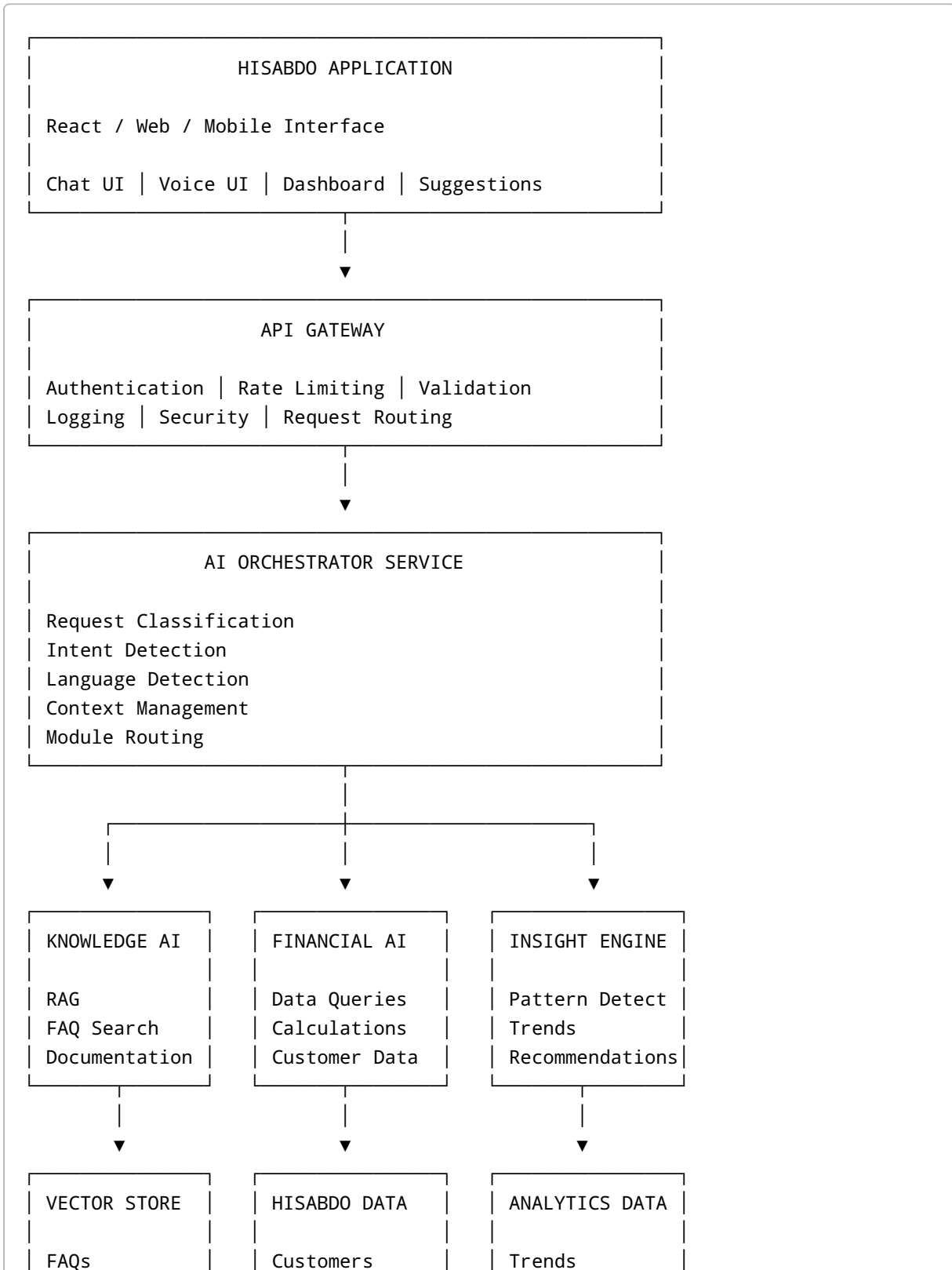
Ali par mera kitna paisa baqi hai?

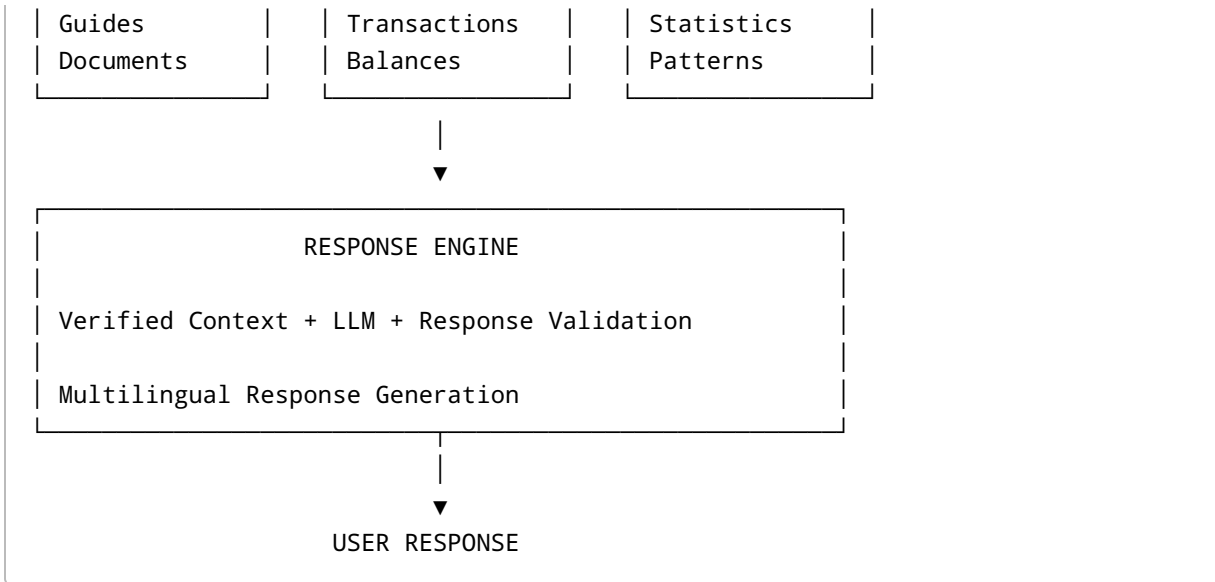
Arabic:

كم المبلغ المستحق علي من علي؟

The system will detect the input language and respond in the same language whenever possible.

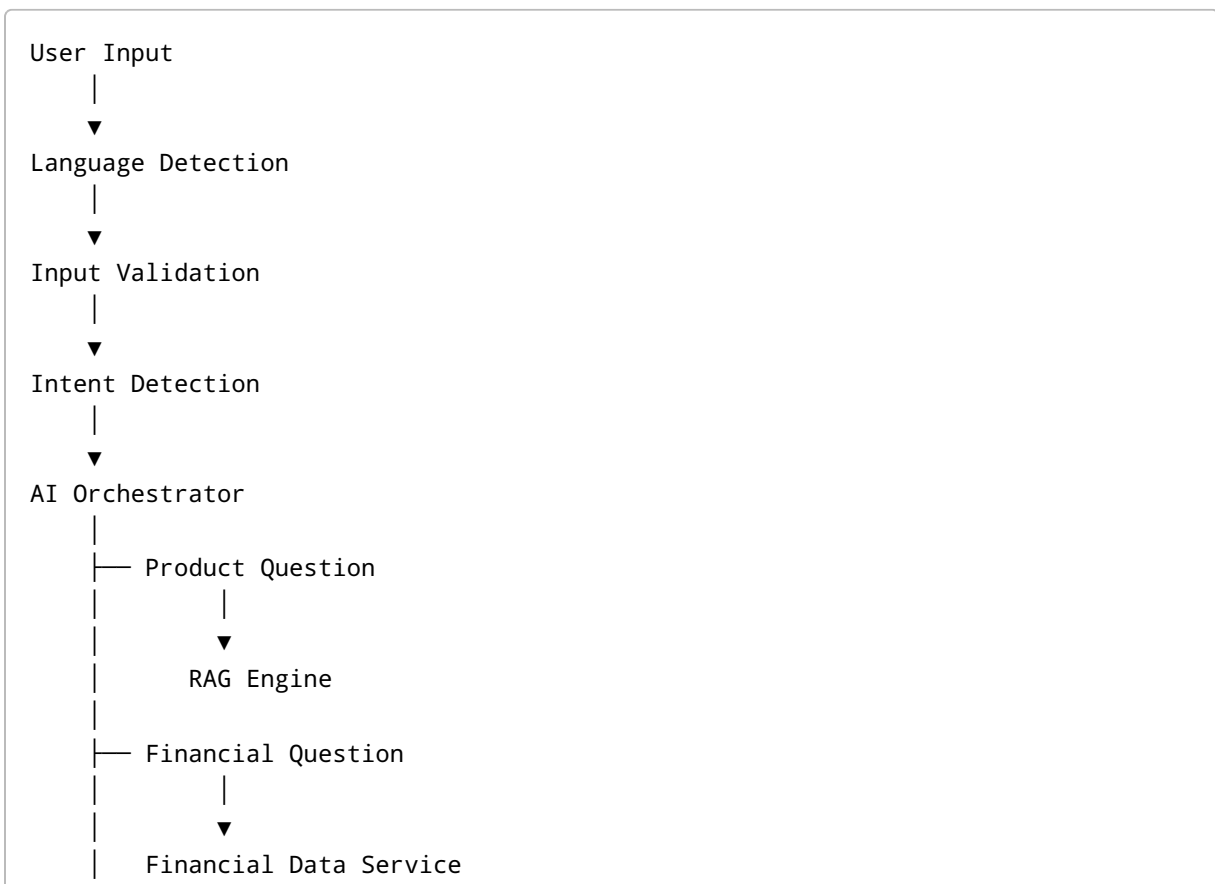
5. Complete End-to-End Architecture

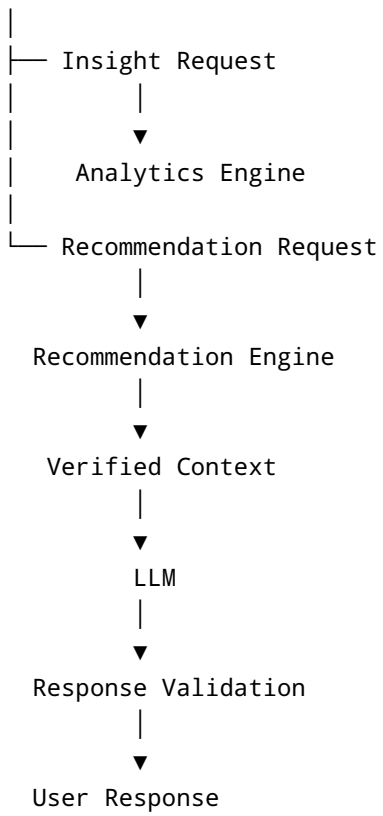




6. AI Request Processing Flow

Every user request will follow this pipeline:





7. AI Orchestrator

The AI Orchestrator is the central intelligence router.

Its responsibility is to determine:

What type of request is this, and which system should handle it?

Example input:

How much does Ahmed owe me?

Classification:

Intent:
CUSTOMER_BALANCE_QUERY

Entity:
Ahmed

Required System:
Financial Data Service

Response Type:
Verified Financial Answer

Example input:

How do I export a report?

Classification:

Intent:
PRODUCT_HELP

Required System:
RAG Knowledge Base

Response Type:
Documentation Answer

8. Intent Categories

The initial intent system will support:

PRODUCT_INFORMATION
FAQ_QUERY
FEATURE_GUIDANCE
CUSTOMER_BALANCE_QUERY
TRANSACTION_QUERY
EXPENSE_QUERY
RECEIVABLE_QUERY
PAYABLE_QUERY
BUSINESS_INSIGHT
ANALYTICS_QUERY
RECOMMENDATION_QUERY
REPORT_HELP
BACKUP_HELP
LANGUAGE_SUPPORT
GENERAL_CONVERSATION
UNKNOWN

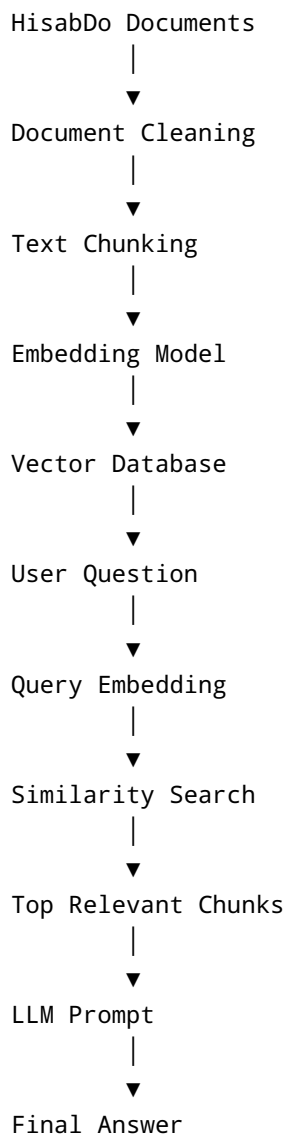
9. RAG Architecture

RAG stands for:

Retrieval-Augmented Generation

The system first retrieves relevant HisabDo information and then allows the LLM to generate an answer based on that verified context.

RAG Pipeline



10. Knowledge Base Data Sources

The knowledge base may include:

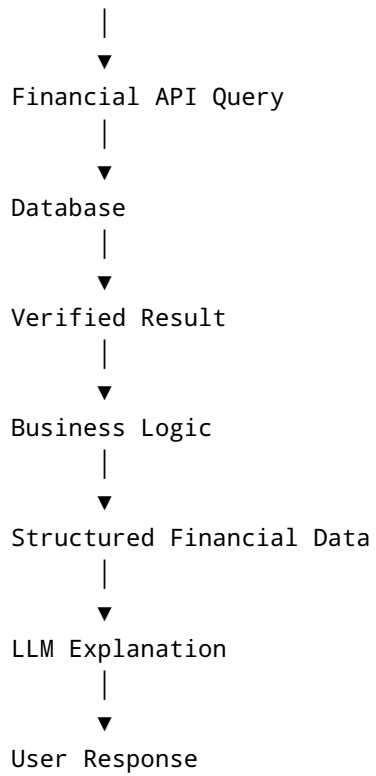
```
/knowledge-base
|
├── faqs
|   ├── general.md
|   ├── accounts.md
|   ├── transactions.md
|   └── backup.md
|
├── features
|   ├── khata.md
|   ├── customers.md
|   ├── analytics.md
|   ├── pdf-export.md
|   └── voice-entry.md
|
├── guides
|   ├── getting-started.md
|   ├── managing-customers.md
|   └── expense-tracking.md
|
└── blogs
    ├── daily-khata.md
    └── digital-ledger.md
```

11. Financial AI Architecture

Financial AI must not directly generate critical numbers.

The system architecture will be:

```
User Question
  |
  ▼
Intent Detection
  |
  ▼
Entity Extraction
```



Example:

User:

Ali ka kitna udhaar hai?

System:

```
Intent:
CUSTOMER_BALANCE_QUERY

Entity:
Ali

API:
GET /api/ai/financial/customer-balance

Result:
{
  "customer": "Ali",
  "balance": 25000,
```

```
"currency": "PKR"
}
```

AI Response:

Ali ka current outstanding balance PKR 25,000 hai.

The amount comes from verified application data.

12. Financial API Layer

Recommended APIs:

Chat API

```
POST /api/ai/chat
```

Request:

```
{
  "message": "Ali ka kitna udhaar hai?",
  "language": "roman-urdu",
  "conversationId": "conversation_id"
}
```

Response:

```
{
  "type": "financial_answer",
  "language": "roman-urdu",
  "message": "Ali ka current outstanding balance PKR 25,000 hai.",
  "data": {
    "customer": "Ali",
    "balance": 25000,
    "currency": "PKR"
  }
}
```

Knowledge Search API

```
POST /api/ai/knowledge/search
```

Request:

```
{
  "query": "How does backup work?"
}
```

Financial Insights API

```
GET /api/ai/insights
```

Example response:

```
{
  "insights": [
    {
      "type": "expense_increase",
      "message": "Expenses increased compared with the previous month."
    }
  ]
}
```

Recommendations API

```
GET /api/ai/recommendations
```

AI Health API

```
GET /api/ai/health
```

13. Multilingual Architecture

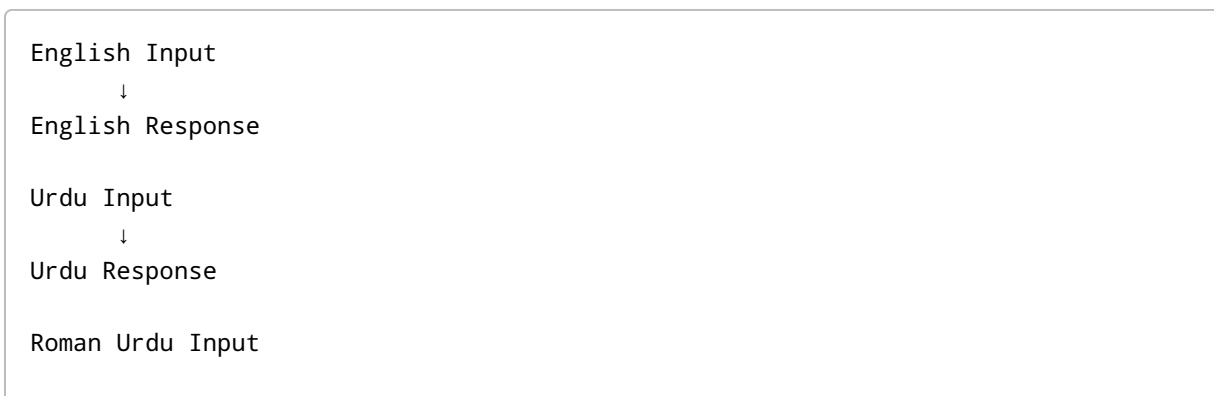
The multilingual system will follow:



Language handling will follow this principle:

Input Language = Output Language

Example:



↓
Roman Urdu Response

Roman Urdu requires special handling because spelling may vary.

Examples:

```
kitna
kitnaaa
kitny
kitnay

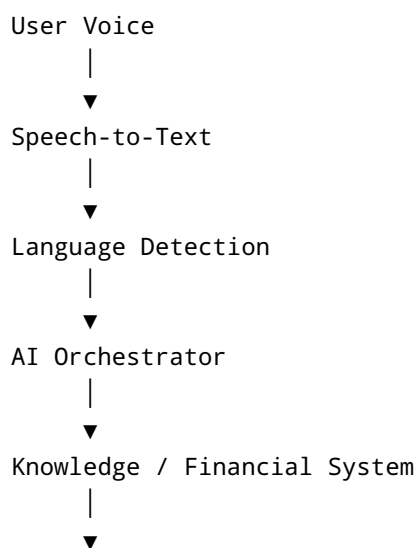
paisa
pese
paisay
paisa

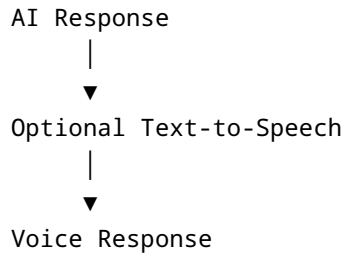
udhaar
udhar
udharr
```

The intent layer should normalize such variations before processing.

14. Voice AI Architecture

HisabDo already supports voice interaction. The AI layer can extend this capability.





Example:

Ali ka kitna paisa rehnda ae?

The system:

1. Converts voice to text.
2. Detects the language.
3. Extracts the customer.
4. Queries verified financial data.
5. Generates the answer.

15. Recommendation Engine

The recommendation system will initially use a hybrid approach.

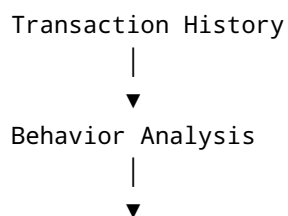
Phase 1

Rule-based recommendations:

```
IF outstanding_balance > threshold  
THEN suggest payment follow-up
```

Phase 2

Pattern-based recommendations:



Pattern Detection



Recommendation

Phase 3

Machine learning recommendations:

Historical User Behavior



Feature Engineering



Recommendation Model



Personalized Suggestions

16. AI Insight Engine

The AI Insight Engine will use deterministic analytics first.

Example analysis:

Current Month Expenses



Previous Month Expenses



Percentage Difference



Insight Generator



AI Explanation

Example:

January Expenses: 50,000
February Expenses: 60,000

Increase:
20%

The application calculates:

$$(60000 - 50000) / 50000 * 100$$

Then the AI explains the result.

17. Recommendation and Insight Examples

The system may generate:

- Your expenses increased this month.
- You have multiple customers with pending balances.
- A customer has not made a payment recently.
- Most of your expenses belong to one category.
- Your highest-value customer generated the largest transaction volume.
- Consider generating a monthly report.

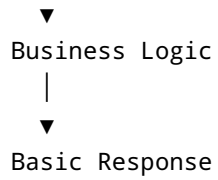
18. Offline-First AI Architecture

HisabDo follows an offline-first approach.

The AI architecture should therefore support multiple modes.

Mode 1: Offline Lite AI

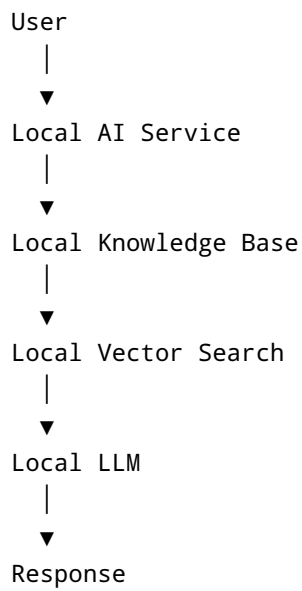
```
User
  |
  ▼
Local Intent Rules
  |
  ▼
Local Database
  |
```



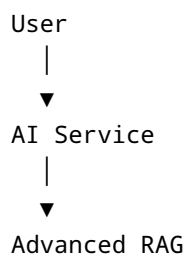
Capabilities:

- Balance lookup
- Basic financial insights
- Feature suggestions
- Rule-based assistance

Mode 2: Local AI



Mode 3: Advanced AI





The architecture allows future deployment flexibility.

19. Recommended Free Technology Stack

AI Service

Python
FastAPI

Local LLM

Ollama

Potential model categories:

Small instruction model
Multilingual model
Lightweight local model

The final model should be selected after benchmarking:

- Response quality
- Multilingual support
- Roman Urdu understanding
- RAM usage
- Response speed

Embeddings

Sentence Transformers

Embeddings should support multilingual semantic search.

Vector Database

For initial development:

FAISS

For future production:

Qdrant

The abstraction layer should allow migration without changing the entire application.

Backend

Node.js
Express.js

Existing HisabDo backend communicates with:

FastAPI AI Service

Frontend

React

Database

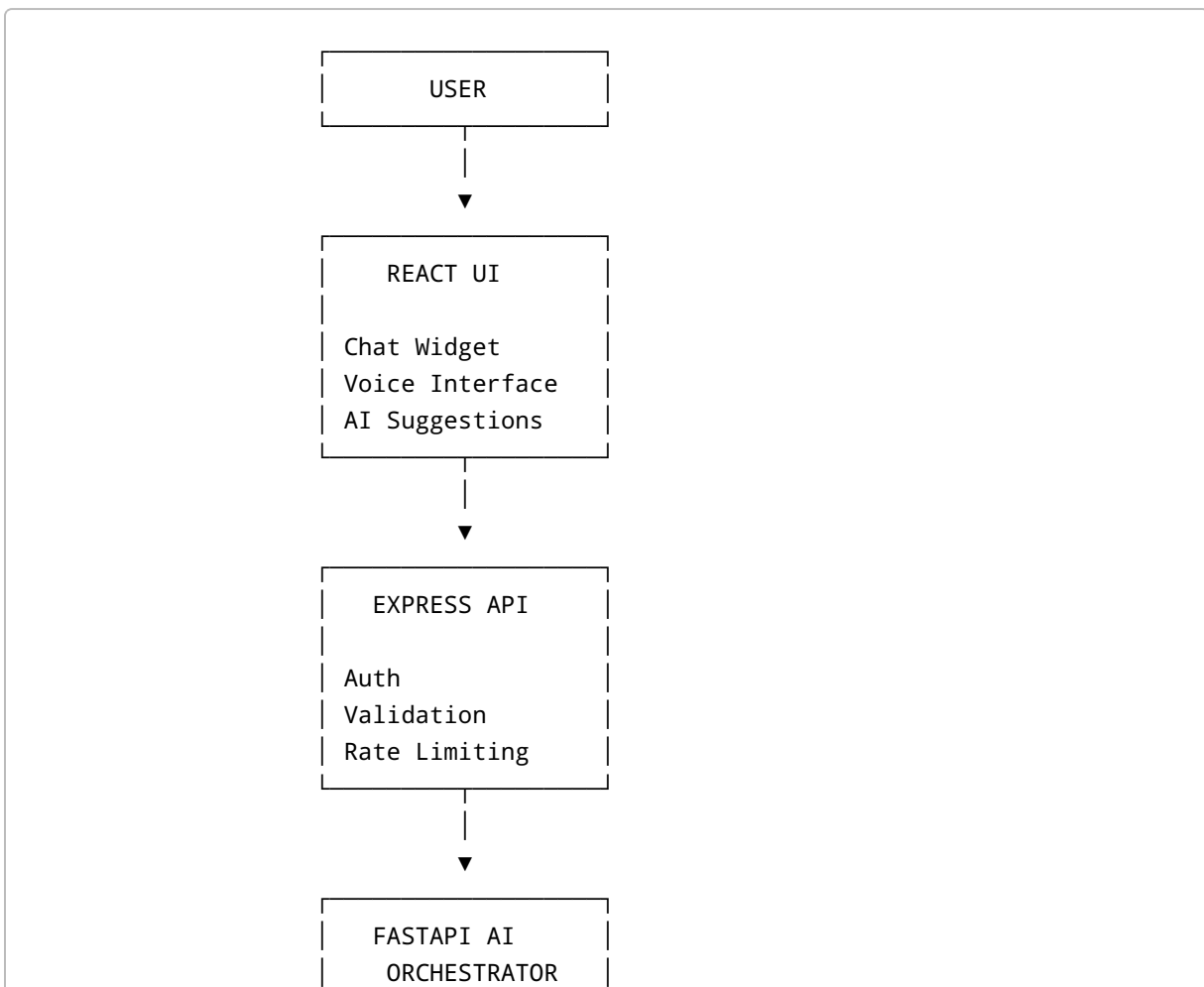
Existing application database:

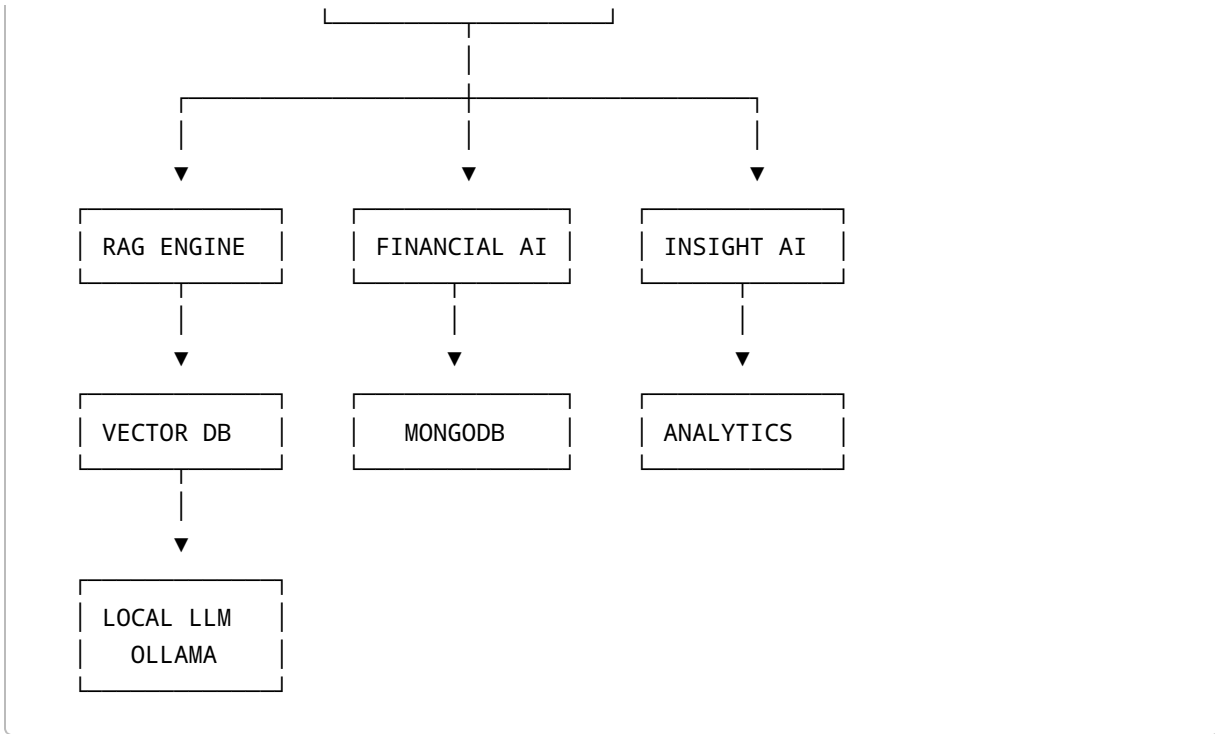
MongoDB

AI-specific data may include:

Vector Store
Conversation Metadata
Feedback Data
Evaluation Data

20. Complete System Architecture





21. Repository Architecture

```
hisabdo-ai/
|
|-- frontend/
|   |-- chatbot-ui/
|   |-- voice-ui/
|   |-- suggestions/
|
|-- backend/
|   |-- controllers/
|   |-- routes/
|   |-- middleware/
|   |-- services/
|
|-- ai-service/
|   |-- app/
|   |   |-- api/
|   |   |-- core/
|   |   |-- orchestrator/
|   |   |-- rag/
|   |   |-- financial/
|   |   |-- insights/
```



```
| | | | | recommendations/
| | | | | multilingual/
| | | | | └─ utils/
| | | | |
| | | | | knowledge-base/
| | | | | └─ faqs/
| | | | | └─ guides/
| | | | | └─ documents/
| | | | |
| | | | | models/
| | | | | tests/
| | | | | └─ requirements.txt
| | | | |
| | | | | docs/
| | | | | └─ architecture.md
| | | | | └─ api-documentation.md
| | | | | └─ ai-roadmap.md
| | | | | └─ security.md
| | | | |
| | | | | └─ README.md
```

22. Security Architecture

Because HisabDo manages financial information, AI security is essential.

Rules:

1. Never send another user's data to a user.
 2. Always verify user identity.
 3. Filter data according to user permissions.
 4. Do not expose raw database access to the LLM.
 5. Use controlled tools and APIs.
 6. Validate all AI-generated actions.
 7. Log important AI operations.
 8. Prevent prompt injection where possible.
 9. Minimize unnecessary financial data exposure.
-

23. AI Safety Rules

The AI must:

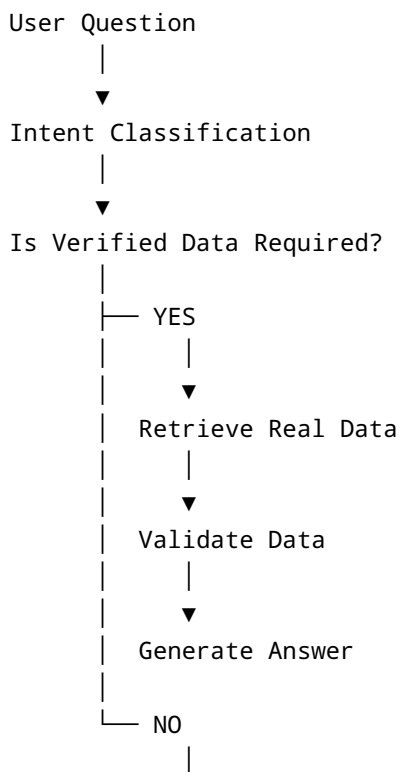
- Never invent financial balances.
- Never perform database queries directly.
- Never access unauthorized user data.
- Never execute financial actions without validation.
- Never expose confidential system prompts.
- Clearly distinguish between calculated facts and general suggestions.

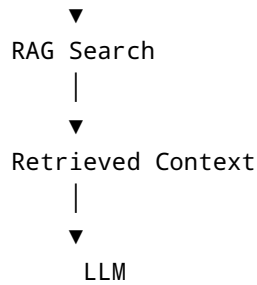
The LLM acts as:

Language Understanding Layer
+
Explanation Layer

It does not become the source of truth.

24. Anti-Hallucination Architecture





25. Conversation Memory

The system should support controlled conversation memory.

Example:

User:

Ali ka balance batao.

System:

Ali ka outstanding balance PKR 25,000 hai.

User:

Uski last transaction kab hui thi?

The system understands:

"Uski" = Ali

Memory should store:

- Conversation ID
- Recent context
- User language
- Selected customer context
- Session metadata

Sensitive financial information should not remain permanently in unnecessary memory.

26. API Response Architecture

Standard response:

```
{
  "success": true,
  "requestId": "unique_request_id",
  "type": "financial_answer",
  "language": "roman-urdu",
  "message": "Ali ka current outstanding balance PKR 25,000 hai.",
  "source": {
    "type": "verified_financial_data"
  },
  "timestamp": "2026-08-19T00:00:00Z"
}
```

27. AI Feedback System

Every AI response may allow feedback:

Helpful
Not Helpful

Feedback can be stored as:

Question
Response
Intent
Language
Helpful/Not Helpful
Feedback Reason
Timestamp

This data can improve:

- Prompts
- RAG quality
- Knowledge base
- Intent detection
- Future fine-tuning

28. Team Responsibilities

Taha — AI/ML Team Lead and AI Architect

Responsibilities:

- Complete AI architecture
- AI strategy
- Technical decisions
- API architecture
- AI orchestration
- Integration planning
- Security review
- Code review
- Team coordination
- Final AI roadmap
- Documentation review

Omesha — AI Chatbot Engineer

Responsibilities:

- Chatbot service
- LLM integration
- Chat API
- Conversation handling
- Intent detection integration
- Response generation
- Prompt management

Abbas — RAG and Knowledge Base Engineer

Responsibilities:

- Knowledge base preparation
- Document cleaning
- Chunking
- Embeddings
- Vector database
- Similarity search
- RAG pipeline
- RAG evaluation

Nobita — AI Research and Data Engineer

Responsibilities:

- FAQ collection
 - AI use-case research
 - Multilingual test cases
 - Recommendation research
 - AI evaluation dataset
 - User query dataset
 - Documentation
 - AI testing
-

29. Day 15–19 Implementation Plan

Day 15 — Architecture and Research

Tasks:

- Analyze HisabDo
- Define AI use cases
- Finalize architecture
- Finalize repository structure
- Collect FAQs
- Prepare chatbot questions

Deliverables:

- AI Architecture
 - AI Use Cases
 - Knowledge Base Structure
-

Day 16 — Chatbot and Knowledge Base

Tasks:

- Setup AI service
- Setup FastAPI
- Prepare Ollama environment
- Prepare knowledge base
- Start document chunking
- Create embeddings

Deliverables:

- AI service running
 - Knowledge base prototype
-

Day 17 — RAG Integration

Tasks:

- FAISS integration
- Similarity search
- RAG pipeline
- Chat API
- Multilingual testing

Deliverables:

- Working FAQ Assistant
-

Day 18 — Financial Intelligence and Recommendations

Tasks:

- Financial intent design
- Financial API tools
- Insight rules
- Recommendation engine
- Anti-hallucination logic

Deliverables:

- Financial AI prototype
 - Insight prototype
-

Day 19 — Integration and Final Report

Tasks:

- End-to-end testing
- API testing
- Multilingual testing
- Performance testing
- Documentation

- Demo preparation

Deliverables:

- Complete AI prototype
 - Architecture documentation
 - API documentation
 - AI roadmap
 - Team report
-

30. Development Priority

The system should be developed in this order:

```
PHASE 1
AI Architecture
    ↓
PHASE 2
FAQ Knowledge Base
    ↓
PHASE 3
RAG Assistant
    ↓
PHASE 4
Multilingual Support
    ↓
PHASE 5
Financial Intent Detection
    ↓
PHASE 6
Financial Data Tools
    ↓
PHASE 7
Business Insights
    ↓
PHASE 8
Recommendations
    ↓
PHASE 9
Voice AI
    ↓
PHASE 10
Offline AI Optimization
```

31. Success Metrics

The AI system should be measured using:

RAG Metrics

- Retrieval accuracy
- Relevant answer percentage
- Unsupported answer percentage

Chatbot Metrics

- Intent detection accuracy
- Response quality
- Average response time
- User feedback

Multilingual Metrics

- Urdu accuracy
- English accuracy
- Hindi accuracy
- Arabic accuracy
- Roman Urdu understanding

System Metrics

- API response time
 - Error rate
 - AI service uptime
 - Vector search speed
-

32. Final Architecture Principle

The complete HisabDo AI system follows one major principle:

AI should understand and explain.
Verified systems should calculate and decide.

The architecture therefore combines:

AI
+
RAG
+
Verified Financial Data
+
Business Logic
+
Multilingual Understanding
+
Offline-First Design
+
User Privacy

This creates a unique AI system that is not simply a chatbot.

It becomes:

HisabDo AI Copilot

An intelligent financial assistant designed to understand:

- The product
- The user
- The user's language
- The user's financial questions
- The user's business patterns

while maintaining:

- Accuracy
- Privacy
- Security
- Offline compatibility
- Cost optimization
- Scalability

33. Final Project Vision

HISABDO AI COPILOT

Understand User Questions

+
Understand Multiple Languages
+
Search Verified Knowledge
+
Analyze Financial Data Safely
+
Generate Business Insights
+
Provide Smart Recommendations
+
Support Voice Interaction
+
Follow Offline-First Principles
=
A Complete AI Financial Assistant

Conclusion

The proposed architecture positions HisabDo beyond a traditional digital ledger application.

Instead of implementing a simple chatbot, this architecture creates a complete AI ecosystem consisting of:

- AI Chatbot
- RAG FAQ Assistant
- Financial Intelligence Layer
- Smart Insight Engine
- Recommendation Engine
- Multilingual AI Layer
- Voice AI Layer
- AI Orchestrator
- Secure API Layer
- Offline-First AI Strategy

The system is designed to start with a practical, free and cost-optimized prototype while remaining scalable for future production deployment.

HisabDo AI Copilot represents the next evolution of intelligent, multilingual and accessible financial management.